

Строки и ввод пользователя

Работать со строками и пользовательским вводом: очищать текст, разбивать его, преобразовывать числа и формировать понятный вывод.

DAY 13

STRING

concatenation

f-string

input

strip

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

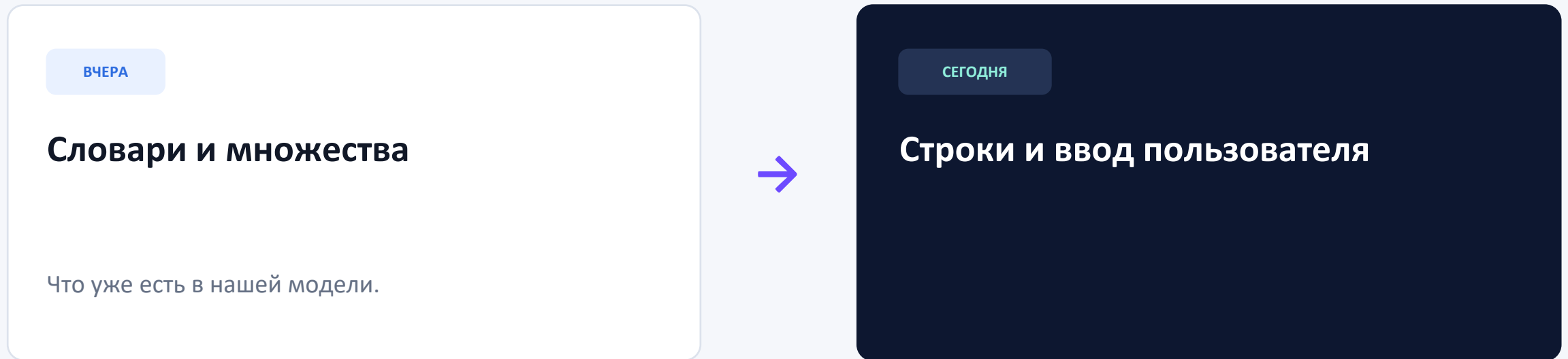
Конкретные действия вместо списка терминов.

- 01 Работать со строками и пользовательским вводом: очищать текст, разбивать его, преобразовывать числа и формировать понятный вывод.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Словари и множества. Сегодня мы добавляем следующий слой: строки и ввод пользователя.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Почти любой интерфейс получает данные как текст. Нужно уметь превращать сырой ввод пользователя в данные, с которыми программа может работать.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Строка похожа на ленту символов: её можно очистить по краям, разрезать по разделителю и собрать обратно в другой форме.

ПЕРЕВОД В IT

string

последовательность текстовых символов

concatenation

соединение строк в одну

f-string

строка с подстановкой выражений в фигурных скобках

Слой 1: string • concatenation • f-string

Сначала смысл, затем профессиональный термин.

**string**

последовательность текстовых символов

**concatenation**

соединение строк в одну

**f-string**

строка с подстановкой выражений в фигурных скобках

Слой 2: input • strip • split

Сначала смысл, затем профессиональный термин.

**input**

функция чтения строки, введённой пользователем

**strip**

метод удаления пробелов и переносов по краям

**split**

метод разделения строки на список частей

Слой 3: substring • delimiter

Сначала смысл, затем профессиональный термин.

**substring**

непрерывная часть строки

**delimiter**

символ или строка, разделяющие части текста

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: formatting • Unicode

Сначала смысл, затем профессиональный термин.

**formatting**

подстановка и оформление значений в строке

**Unicode**

стандарт представления символов разных языков

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.

```
name = input("Имя: ").strip()
minutes = int(input("Минуты: "))
print(f"{name}, ты занимался {minutes} минут")
```

day13_step1

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
name = input("Имя: ").strip()
```

Сохраняем значение или результат вычисления под понятным именем.

```
minutes = int(input("Минуты: "))
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(f"{name}, ты занимался
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
name = input("Имя: ").strip()
minutes = int(input("Минуты: "))
print(f"{name}, ты занимался {minutes} минут")
```

РЕЗУЛЬТАТ

Имя: Anna
Минуты: 45
Anna, ты занимался 45 минут

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
skills_text = "Python, Git, SQL"  
skills = skills_text.split(", ")  
print(skills)
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

```
['Python', 'Git', 'SQL']
```

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

```
age = input("Возраст: ")  
print(age + 5)
```

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

input() всегда возвращает строку. Перед арифметикой нужно преобразовать значение в int или float.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.



fixed

```
age = int(input("Возраст: "))  
print(age + 5)
```

ПРАВИЛО

**Не переписывай всё
решение. Найди
минимальное
осмысленное
исправление.**

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Получить имя и список навыков через запятую, очистить пробелы и вывести персональное сообщение.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

1 Читаем имя и применяем `strip()`.

2 Читаем строку навыков.

3 Разбиваем по запятой.

4 Очищаем каждый элемент.

5 Печатаем имя и готовый список.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
name = " Anna ".strip()
raw = "Python, Git, SQL"
skills = [item.strip() for item in raw.split(",")]
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



solution_final

```
name = " Anna ".strip()
raw = "Python, Git, SQL"
skills = [item.strip() for item in raw.split(",")]
print(name)
print(skills)
```

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Anna
['Python', 'Git', 'SQL']

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Попроси пользователя ввести город и год рождения, затем выведи аккуратную f-строку.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Год — число, город — текст. Не забудь `strip()` для текстового ввода.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
city = " Tallinn ".strip()
year = int("2000")
print(f"Город: {city}; год рождения: {year}")
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Город: Tallinn; год рождения: 2000

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

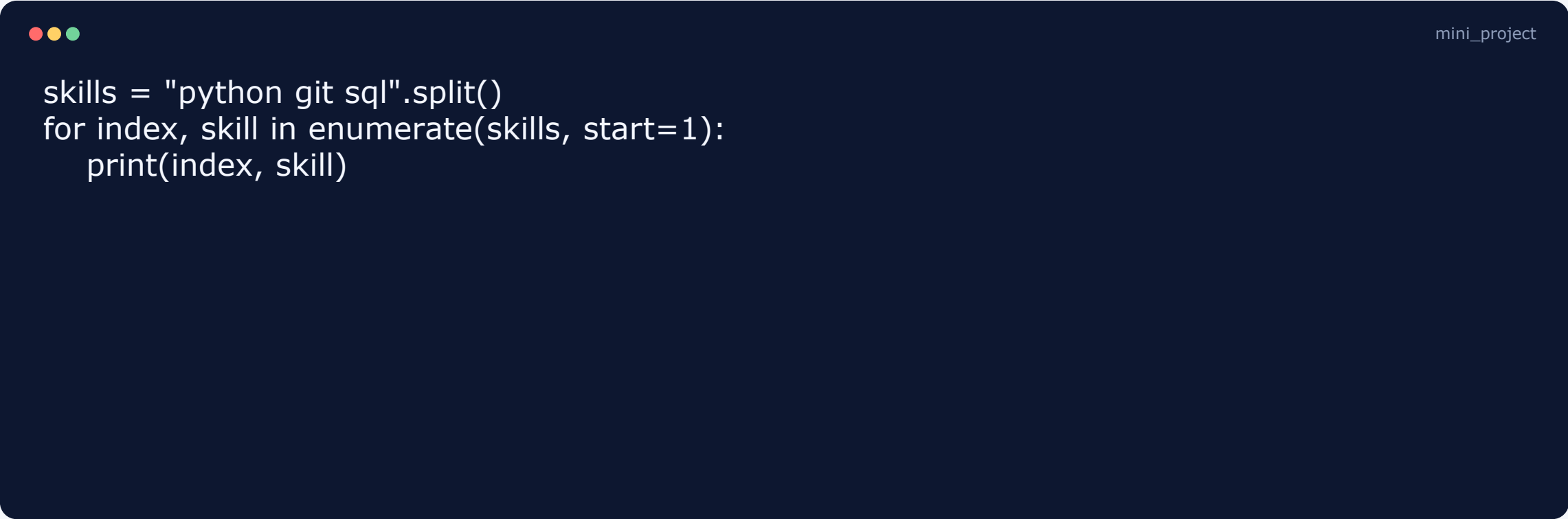
ЗАДАНИЕ

Разбери строку "python git sql" на слова и выведи каждое с порядковым номером.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.

A dark-themed terminal window with a title bar at the top containing three colored circles (red, yellow, green) on the left and the text 'mini_project' on the right. The terminal contains the following Python code:

```
skills = "python git sql".split()
for index, skill in enumerate(skills, start=1):
    print(index, skill)
```

```
skills = "python git sql".split()
for index, skill in enumerate(skills, start=1):
    print(index, skill)
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

```
1 python
2 git
3 sql
```

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Забывать, что `input()` возвращает `str`.

02

Не очищать лишние пробелы.

03

Путать `split()` и `join()`.

04

Собирать длинные строки сложением вместо `f`-строк.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «string» и какую проблему это понятие помогает описать?

2

Чем «string» отличается от «input»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **string — последовательность текстовых символов.**

2 string: последовательность текстовых символов. input: функция чтения строки, введённой пользователем.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3

Имя: Анна

Минуты: 45

Анна, ты занимался 45 минут

4

`input()` всегда возвращает строку. Перед арифметикой нужно преобразовать значение в `int` или `float`.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ **Работать со строками и пользовательским вводом: очищать текст, разбивать его, преобразовывать числа и формировать понятный вывод.**
- ✓ Читать и объяснять пример темы «Строки и ввод пользователя».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Функции

Завтра: Функции. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.